

TP1

EXERCICE 1 :

On considère le script suivant :

```
for i in range(len(t)-1):
    if t[i] > t[i+1]:
        t[i] , t[i+1] = t[i+1] , t[i]
```

1. Quelle est la complexité de cet algorithme ?
2. Quelle est le contenu de la liste `t = [4, 3, 2, 1]` après l'exécution de cet algorithme ?

EXERCICE 2 :

On considère le script suivant :

```
for i in range(len(t)):
    for j in range(i):
        print(t[j], end= "_")
```

3. Quelle est la complexité de cet algorithme ?
4. Quelle est l'affichage de cet algorithme pour `t = [1, 4, 2, 3]`

EXERCICE 3 :

5. Écrire une fonction nommée `recherche_lineaire(liste,valeur)` qui recherche une valeur dans une liste non triée. Chaque élément va être comparé à `valeur`

Cette fonction renverra l'indice de la valeur si elle est trouvée et `False` sinon

6. Quelle est la complexité de cet algorithme ?
7. L'algorithme linéaire trouvera plus facilement les éléments : (entourer la bonne réponse) :
 - Les plus petits
 - Les plus grands
 - Au début de la liste

EXERCICE 4 :

On considère l'algorithme de dichotomie suivant :

```

1  def recherche_dichotomique(L:list,valeur:int):
2      ''' Algorithme de recherche dichotomique '''
3      i_deb = 0
4      i_fin = len(L) - 1
5      while i_fin - i_deb >= 0:
6          i_cen = (i_deb + i_fin) // 2
7          if L[i_cen] == valeur:
8              return True
9          elif valeur > L[i_cen]:
10             i_deb = i_cen + 1
11          else:
12             i_fin = i_cen - 1
13      return False

```

8. Écrire la trace de cet algorithme lors de l'appel suivant :

```
recherche_dichotomie([1, 6, 15, 20, 24], 8)
```

Vous indiquerez

- Le numéro de la ligne active
- les valeurs des différentes variables lorsqu'elles sont modifiées ou utilisées L'état logique du WHILE, IF et ELIF (True ou False) lorsque ces lignes seront activées

Ligne	<u>i_debut</u>	<u>i_fin</u>	<u>i_centre</u>	L[i_centre]	While	IF	ELIF

9. Quelle est la complexité de cet algorithme?
10. Quand on multiplie par deux la taille du tableau, combien fait-on d'itérations en plus?

EXERCICE 5 :

L'objectif de cet exercice est de déterminer la valeur qui annule une fonction, c'est à dire la valeur de x telle que $f(x) = 0$

- Télécharger le fichier `tp_note.py`
- Renommer ce fichier `tp_note_NOM.py`
- A la fin du TP vous déposerez ce fichier à l'endroit indiqué

1. Créer une fonction `creation_abscisses(a:int,b:int)->list`: qui renvoie une liste contenant tous les entiers compris entre a et b inclus.

```
assert creation_abscisses(-3,3) == [-3, -2, -1, 0, 1, 2, 3]
```

2. Créer une fonction `creation_ordonnees(abscisses:list,fonction)->list`: qui renvoie une liste contenant toutes les images de chaque abscisses contenues dans la liste `abscisses`.

```
assert creation_ordonnees([-3,-2,-1,0,1,2,3],f) == [-35,-12,-3,-2,-3,0,13]
assert creation_ordonnees([-3,-2,-1,0,1,2,3],g) == [-89,-28,-5,-2,-1,16,67]
```

3. Modifier la fonction `recherche_dichotomique(L:list,valeur:int)` afin qu'elle renvoie :

- L'indice où se trouve `valeur` si cet élément est dans la liste `L`
- False si cet élément n'appartient pas à la liste.

4. Écrire une fonction `recherche_zero(a:int,b:int,fonction)` qui renvoie l'indice de l'abscisse x telle que $f(x) = 0$ en utilisant la fonction de recherche dichotomique.

```
assert recherche_zero(-3,3,f) == 5
```

5. Écrire une fonction `recherche_dichotomique2(L:list,valeur:int)` qui renvoie

- L'indice où se trouve `valeur` si cet élément est dans la liste `L`
- L'indice du dernier élément plus petit que `valeur`

```
assert recherche_dichotomique2([-2,-1,2,4],0) == 1
```